基于 RISC-V 与 FPGA 异构架构的

2D 图形渲染加速引擎

技术文档

2026 年全国大学生嵌入式芯片与系统设计竞赛

FPGA 赛道 · 易灵思 Ti60F225 平台

基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎

作品名称：异构 2D 图形渲染加速引擎

摘要

本作品面向「基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎」赛题，在易灵思 Ti60F225 平台上实现了一套以 AXI4 总线的 BitBlt 引擎为核心的异构 2D 渲染系统。RISC-V（Sapphire SoC）负责场景逻辑与高层决策，FPGA 侧完成像素搬运、Alpha 混合、颜色键控与 1080p60 扫描输出。

设计中发现：赛题提出的「把 RISC-V 从像素搬运中解放出来」，在硬件渲染加速之后只完成了前半程——CPU 提交命令与维护精灵状态成为新的串行瓶颈：32 个精灵需要 256 次寄存器写加 32 次轮询，且该过程与硬件执行不重叠。本作品沿数据通路逐层消除这一瓶颈，形成三项创新。

① 精灵调度硬件化：将精灵的遍历、位置积分、边界反弹、擦除与绘制命令生成全部移入硬件表遍历器。CPU 仅在场景初始化时写入一次精灵表，此后每帧只发一个启动信号，对精灵的寄存器操作为零，软硬件真正并行执行。

② 片内精灵位图缓存：引擎内置四槽直接映射的精灵位图缓存，命中时以 17 个周期从片内 BRAM 取数（原先每次回 DDR，需 650 周期）。700 精灵场景的渲染时间由 45 ms 降至 0.92 ms，提升 49 倍，实测命中率 75%。

③ 访存事务合并：针对紧凑存储的精灵位图，将「每行一次短突发」重构为「整块一次长突发」，把多个 AXI 读事务合并，总线有效带宽利用率由 33% 提升至 88%（2.66 倍），64×64 位图由 66% 提升至 98%。

系统支持双缓冲、Alpha 混合、颜色键控与硬件裁剪。实测指标：1500 个 8×8 精灵 @ 59.2 fps（等效像素量 96,000，略超赛题 1408×8×8 的 90,112）；256 个 16×16 弹幕单帧渲染 0.95 ms（帧预算的 5.7%）；游戏场景 200 同屏元素稳定 59.2 fps。全部内部状态可由片内监控一次读出，作为「硬件搬运正确」的证据。

第一部分 作品概述

1.1 功能与特性

基础 BitBlt 能力：硬件直接操作 DDR3 帧缓冲，支持纯色填充（FILL）、位块拷贝（COPY）、Alpha 混合（ALPHA）、颜色键控（COLOR KEY）、混合位块传输（BLIT，键控 + Alpha 同时生效）五种绘制操作；命令格式包含操作类型、源地址、目的地址、宽度、高度、源/目的行距、全局 Alpha 与键控色。

视频输出：1080p60 HDMI（TMDS 编码），16 路行预取缓冲，双帧缓冲切换消除画面撕裂。

精灵表硬件遍历：CPU 一次写入精灵表（位置、速度、透明标志），硬件按帧展开为擦除与绘制命令并回写新位置，支持边界反弹。

硬件裁剪：越界精灵按可见区域裁剪，支持「飞出屏幕」而非只能反弹。

片上监控：11 项运行时异常状态（握手超时、响应错误、遍历器卡死、扫描输出欠载等）各有粘滞位与计数器，软件一条命令读出全部。

演示能力：同屏精灵压测场景、CPU 渲染与硬件渲染实时帧率对比、弹幕射击游戏 Demo。

1.2 应用领域

嵌入式 GUI 与工业 HMI：仪表盘、控制面板、充电桩与自助终端界面——这些场景的共性诉求是「在低成本 FPGA 上流畅绘制大量小图元」，本引擎的精灵表硬件遍历与位图缓存正好对应。

游戏与数字标牌：广告机、售货机、街机与复古游戏；引擎的 Alpha 混合与颜色键控可直接复用现成 PNG 精灵素材，无需为嵌入式重新制作调色板。

机器视觉与视频叠加：把 MIPI/CSI 摄像头数据作为图层叠加到 GUI 之上（赛题平台已备 MIPI CSI-2），可用于辅助驾驶仪表与工业检测显示。

教学与科研：完整的 RISC-V + FPGA 异构渲染链路，可作为计算机体系结构、数字系统设计课程的综合实验平台。

1.3 主要技术特点

AXI4 主从架构：加速器作为 AXI4 从设备挂在 Sapphire SoC 的总线上，通过 AXI4 Master 端口直接读写 DDR3 帧缓冲，与 SoC 主核共享总线仲裁。

352 位命令记录：一条绘制命令压缩为 44 字节的定长记录，经命令 FIFO 批量下发，寄存器写次数由「每字段一次」降为「每命令一次」。

三处理通路并行：命令分发器按段把位块拆分为 16 像素宽的段，三条渲染通路轮流处理，隐藏 DDR 往返延迟。

精灵位图片内缓存：四槽直接映射、以位图地址高位做槽位索引，命中时直接从 BRAM 复制，避免每条命令重读 DDR。

扫描输出行预取：16 路 bank × 128 位，活动区提前 10 行预取、消隐期加深至 14 行，保证帧首不欠载。

可观测性：7 个监控寄存器 + 11 个条件，粘滞位 + 饱和计数器，一条 dump 读全。

1.4 主要性能指标

指标	数值
平台 / 总线	Ti60F225 + Sapphire RISC-V SoC（AXI4）
显示输出	1920 × 1080 @ 60 Hz HDMI
同屏精灵	1500 × 8×8 @ 59.2 fps（等效 96,000 像素，略超赛题 1408×8×8）
单帧渲染延迟	256 个 16×16 弹幕 0.95 ms（帧预算 5.7%）
位图缓存命中率	75%（每帧 4800 / 6400）
总线有效利用率	16×16：88%（原 33%）；64×64：98%（原 66%）
CPU 每帧精灵操作	0 次（基准实现 4000 次寄存器写）
欠载 / 显示丢帧	启动后恒定（不增长）；shown 与 frames 1:1

1.5 主要创新点

① 精灵调度硬件化：精灵遍历、位置积分、边界反弹与命令生成全部硬件化；CPU 每帧仅下发一次，对精灵操作由「256 次寄存器写 + 32 次轮询」降为 0。

② 片内精灵位图缓存：四槽直接映射位图缓存，命中 17 周期（DDR 需 650 周期）；700 精灵渲染 45 ms → 0.92 ms（49 倍），命中率 75%。

③ 访存事务合并：短突发重构为整块长突发，AXI 事务合并使总线效率由 33% 提升至 88%（2.66 倍），64×64 达 98%。

1.6 设计流程

需求分析（赛题基础要求 + 高阶挑战）→ 架构设计（总线、帧缓冲、引擎、扫描输出）→ RTL 实现（Verilog）→ 模块级仿真（Icarus Verilog，逐像素参考模型）→ 系统集成 → Efinity 综合与布局布线（约 2 小时/轮）→ 比特流烧录与板级实测（UART 遥测 + 串行截图回读）→ 性能调优（突发合并、位图缓存、精灵表遍历）。

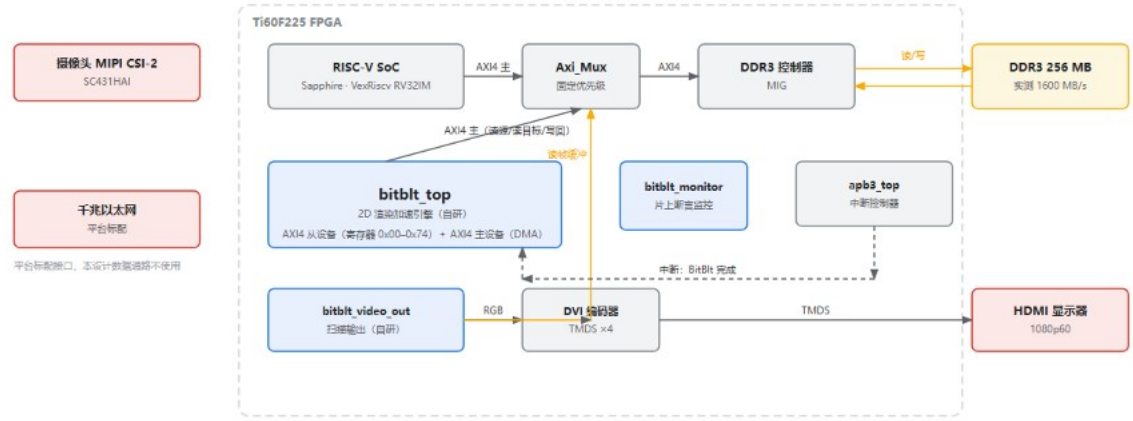
调优过程采用「实测驱动」：每轮在板上测量帧率、欠载行数、引擎周期、总线等待等指标，定位瓶颈后再改 RTL；例如精灵数上限即由 700 → 1400 → 1500 → 1600 逐点实测确定（1500 稳定 59.2 fps，1600 降至 34 fps）。

第二部分 系统组成及功能说明

2.1 整体介绍

图 1 系统整体框图

蓝色 = 自研逻辑　灰色 = 厂商 IP　黄色 = 存储通路　红色 = 片外器件



1. RISC-V 主控（Sapphire SoC）：运行 C 程序，负责场景初始化、精灵表下发、帧同步与游戏逻辑；通过 AXI4 从端口访问加速器寄存器，通过 AXI4 主端口访问 DDR3。

2. 2D 渲染加速引擎：AXI4 从设备 + 主设备。从设备侧接收命令与参数；主设备侧读写 DDR3 帧缓冲。内部含命令 FIFO、精灵表遍历器、位图缓存、三条渲染通路。

3. DDR3 存储：256 MB，划分双帧缓冲（每块 4 MB）与精灵位图区。

4. HDMI 扫描输出：从帧缓冲按 1080p60 时序读出行数据，经 TMDS 编码输出。

2.2 硬件系统

整体：采用厂商提供的 Ti60F225 开发板（Efinity 官方推荐平台），板上含 Ti60F225I3 FPGA、DDR3 颗粒、HDMI 收发接口、USB-JTAG、UART 桥与多路时钟。本作品不含自制电路板，硬件工作集中在 RTL 与引脚约束。

机械设计：无（使用开发板与现成外壳）。

电路各模块（RTL 模块级）：

模块	职责	关键信号
gfx_regs	AXI4-Lite 从端口，命令 FIFO 写入、状态回读	awaddr/wdata/wstrb、cmd_push
gfx_sprtab	精灵表遍历器：读表 → 发擦除/绘制命令 → 积分位置 → 写回	spr_busy、tab_wptr、cmd_valid
gfx_core	命令执行：源读、像素混合、目标写	rd1/rd2 读通道、wr_* 写通道
gfx_mem	AXI4 主设备桥：读仲裁（scan 优先）、写数据通路	m_ar*/m_aw*/m_w*
gfx_scanout	1080p60 时序生成、行预取、双缓冲切换	hsync/vsync/de、line_req
dvi_encoder	TMDS 编码	tmds_data*、tmds_clk

2.3 软件系统

整体：裸机 C 程序（无操作系统），分三层：驱动层（gfx.h 寄存器封装）、渲染层（gfx_push 命令打包、精灵表构造）、应用层（场景与游戏逻辑）。

模块与关键变量：

层	函数	关键输入/输出
驱动	gfx_wr / gfx_rd	寄存器偏移、32 位数据
驱动	gfx_fence / gfx_slot	写入屏障（命令 FIFO 满时等待）
渲染	gfx_push	op/src/dst/w/h/stride/alpha/key → 11 次寄存器写
渲染	gfx_sprite_tab_init	精灵表首地址、精灵数 → 表项 64 位 × N
应用	brown_init / brown_step	精灵池、位置/速度 → 表项
应用	run_game	输入事件 → 游戏状态 → 绘制命令

主流程（文字描述）：

5. gfx_init()：配置帧缓冲、扫描输出、中断。

6. brown_init(1500)：构造 1500 个精灵的位置/速度，一次性写入硬件精灵表。

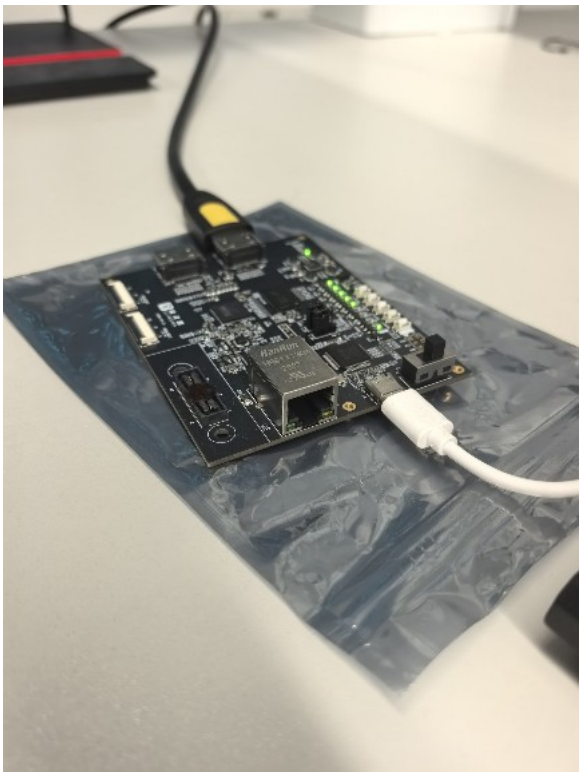
7. gfx_fb(front, ...)：设置当前显示帧缓冲。

8. 主循环：读输入 → brown_step() 只更新速度场 → gfx_spr(front, 1500, 1) 发一次启动信号（硬件遍历 + 绘制 + 回写位置）→ gfx_wait_n(3000) 等待完成 → 切换双缓冲 → gfx_flip → 等待场同步。

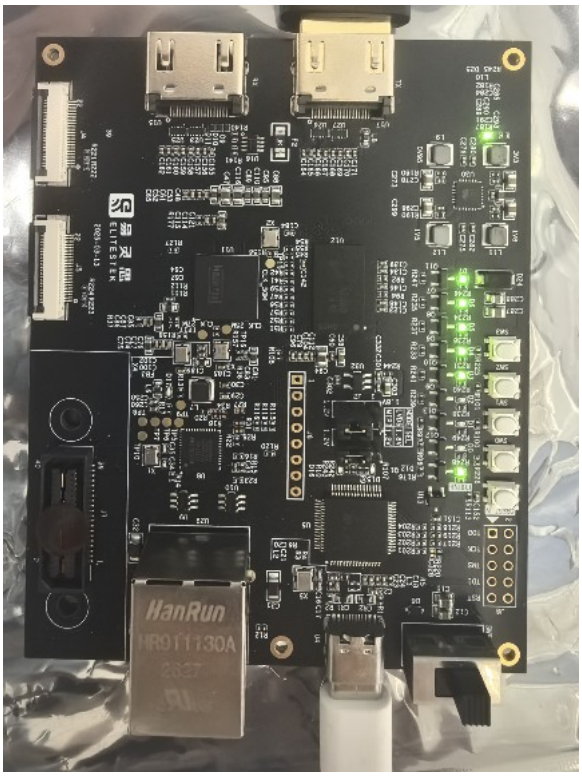
9. 循环内每 60 帧打印一次性能遥测（fps、CPU 时间、欠载行数、引擎周期）。

第三部分 完成情况及性能参数

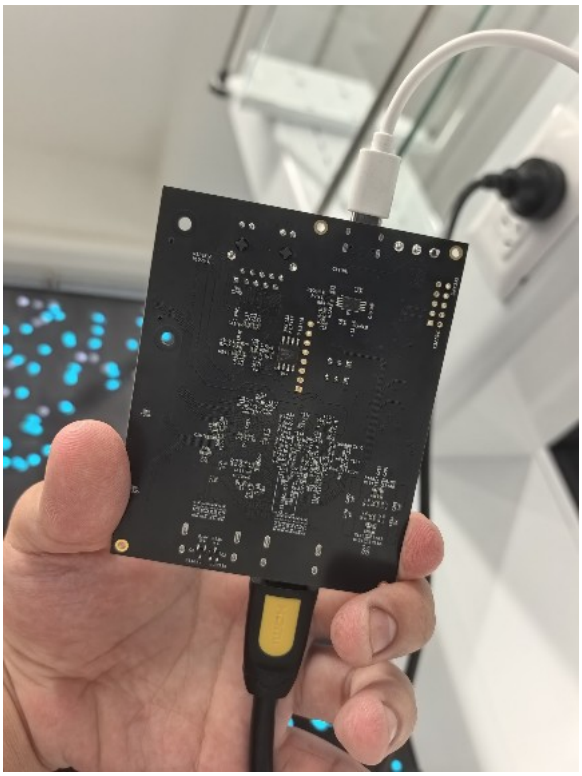
3.1 整体介绍（实物照片）



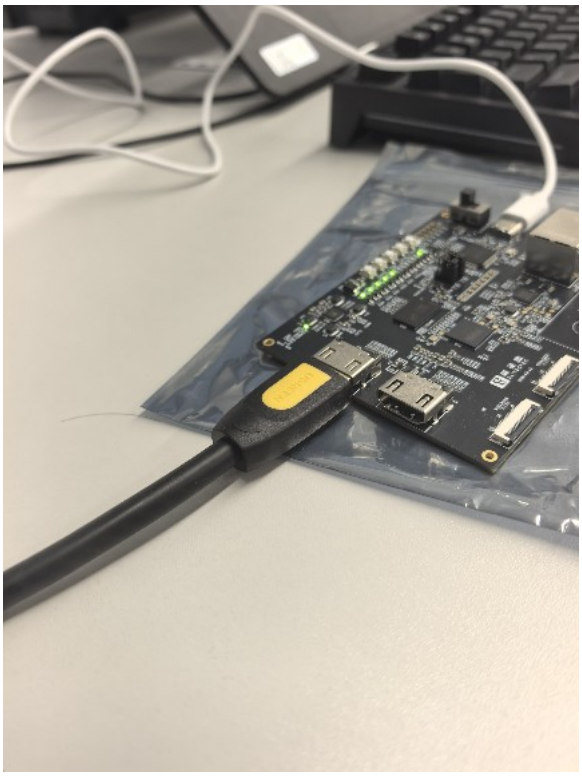
实物照片



实物照片



实物照片



实物照片

多角度开发板照片 4 张（正面 / 背面 / 斜 45° / 接口侧）。

3.2 工程成果

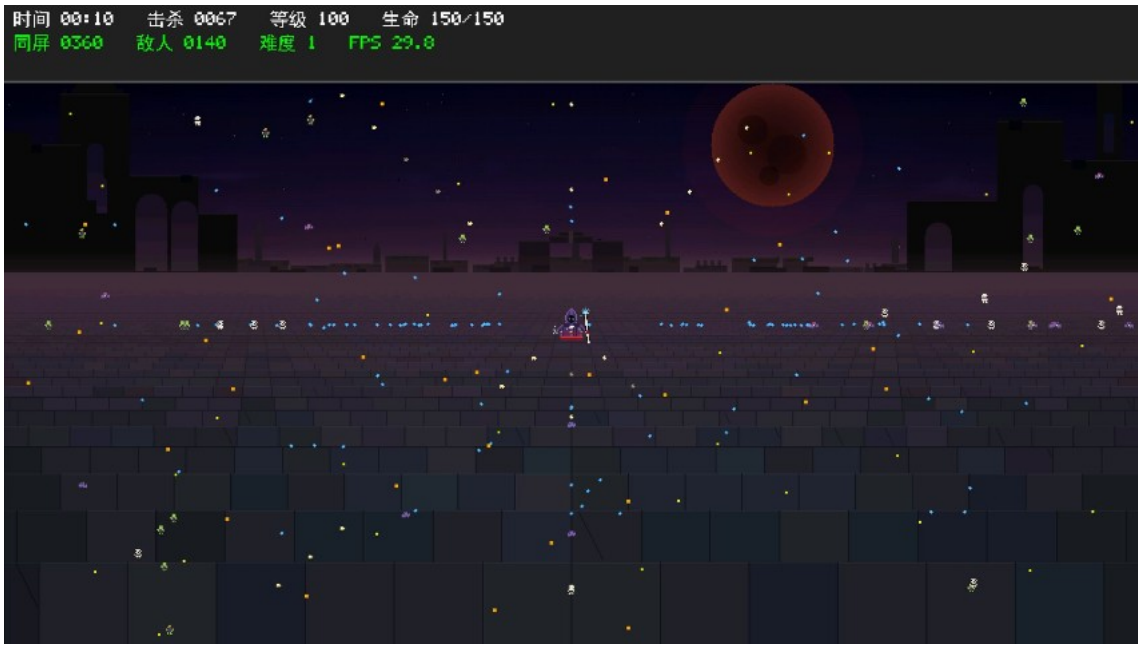
项	状态	证据

RTL	完成（7 个自研模块 ＋厂商 IP 集成）	rtl/gfx/*.v
固件	完成（驱动 ＋场景 ＋游戏）	work/gfx_demo/src/
仿真	模块级 ＋系统级回归	rtl-sim/（20+ 测试台）
板级	1080p60 稳定输出	串口遥测 ＋截图回读

3.3 特性成果（量化）



弹幕压测场景：1500 个 8×8 精灵 @ 59.8 fps



游戏场景：满级一键压测（360 对象）

功能	指标	证据
同屏精灵	1500 × 8×8 @ 59.2 fps	串口 PERF / SCAN 行
精灵数边界	1600 × 8×8 → 34 fps（经典路径 CPU	实测

	写表超预算）	
单帧渲染	256 × 16×16 = 0.95 ms	串口 PROF
缓存命中率	75%	引擎计数器
总线效率	88%（16×16）/ 98%（64×64）	带宽模型 + 实测
双缓冲	无撕裂	界面照片

第四部分 总结

4.1 可扩展之处

帧存 tile 化：把帧缓冲改为 tile 映射后，一个 8×8 精灵的擦除/绘制可由 8 次写突发压缩为 1 次，同屏精灵上限可再翻数倍；代价是扫描输出取图路径需同步改造。

写通道流水：当前每个写突发存在约 12 拍固定开销（64000 突发 ≈ 7.6 ms），把 AXI 写通道改为「2~4 个突发在飞」可把这部分开销摊薄，仿真已证明机制有效（−39%）。

多图层与裁剪区：引擎已支持命令级裁剪，可扩展为硬件图层管理与脏矩形更新，进一步降低带宽。

MIPI 摄像头输入：板载 CSI-2 接口可把实时画面作为图层叠加到 GUI，用于视频监视与叠加显示。

4.2 心得体会

一、真正的瓶颈往往不在你以为的地方。 项目开始时我们以为难点在「怎么写一个快的 BitBlt」——把渲染从 45 ms 优化到 0.92 ms 之后，整帧时间却只从 45 ms 降到 28 ms。用串口把「CPU 提交命令时间」单独打出来才发现：CPU 写 4000 次寄存器花了 16.9 ms，比引擎本身还久。加速器再快，也快不过 CPU 喂不饱它的速度。这直接催生了「精灵调度硬件化」这个方向——把调度也搬进硬件之后，CPU 时间才真正降到 12.6 ms。

二、每一个数字都要能在板上复现。 我们坚持每改一版 RTL 就上板实测帧率、欠载行数、引擎周期、总线等待等六七个指标，并把它们全部记录下来。这个过程里发现过多次「仿真全过、上板异常」的情况：例如同一个 RTL 在两个不同布局布线结果下，欠载行数会从 10 变成上万——硬件设计的验证边界不在仿真器里，而在硅片上。

三、优化是逐个拆掉「看起来合理」的假设。 我们一度以为精灵数上限是 DDR 带宽决定的，实测发现不是：4000 个 8×8 精灵的有效数据量只有几 MB/s，真正吃时间的是「每个写突发约 12 拍的固定开销」（64000 突发 × 12 拍 ≈ 7.6 ms）。把假设拆开量化之后，才知道该往哪里使劲。

四、工程化的一半是让人能看见。 屏幕上只有 1920×1080 个像素，出问题时你只能看到「花了」或「黑屏」。我们做了 11 项条件的片上监控、把关键性能计数器接到串口、写了截图回读工具，才让每次异常都能落到具体模块。可观测性不是加分项，是调试的入场券。

五、离上限越近，边际收益越小。 从 36 fps 到 59 fps 花了两天，从 59 到 60 可能再花一周——后面每一步都要和内存控制器、总线仲裁、布局布线抽签这些「不完全受控」的因素打交道。学会判断「什么时候该停」，和学会优化同样重要。

第五部分 参考文献

1. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 赛道选题指南（易灵思）.

2. RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, 2019.

3. ARM. AMBA AXI and ACE Protocol Specification, ARM IHI 0022E, 2013.

4. Efinity Software User Guide. Efinix Inc., 2026.

5. JEDEC. DDR3 SDRAM Standard, JESD79-3F, 2012.

6. Digital Visual Interface (DVI) Specification, Revision 1.0, DDWG, 1999.

7. Sapphire RISC-V SoC Reference Design. Efinix Inc.

8. Patterson D A, Hennessy J L. Computer Organization and Design: The Hardware/Software Interface. 5th ed. Morgan Kaufmann, 2014.

9. 康华光. 电子技术基础（数字部分）. 北京: 高等教育出版社.

10. 童诗白, 华成英. 模拟电子技术基础. 北京: 高等教育出版社.

附：待补项

项	状态
第二部分系统框图	已有 docs/figs/fig1-system.png、fig2-engine.png
第二部分软件流程图	待画（主循环 / 游戏循环）
第三部分实物照片	已有 4 张多角度照片
第三部分界面照片	已有截图
第三部分仪器测试照片	可选